



学习任务2: Toffee 部分

用 Toffee 搭建验证环境

验证对象的代码和文档位于：[同步 FIFO 的设计规范和代码](#)。

1. 用 toffee-test 管理测试用例

准备工作

在根目录下创建 `pyproject.toml` 文件，内容为：

```
[tool.pytest.ini_options]
pythonpath = "./"
```

如果你对 `pyproject.toml` 在这里的作用感到好奇，请阅读
<https://docs.pytest.org/en/stable/explanation/goodpractices.html>

练习内容

把先前编写的测试用例用 `toffee-test` 进行管理：

1. 创建 `tests` 文件夹
2. 在 `tests` 文件夹下创建 `test_smoke.py` 文件，把先前编写的代码用 `toffee-test` 进行管理。

2. 使用 Bundle 封装 DUT

准备工作

创建 `tests` 和 `bundle` 文件夹，在 `bundle` 文件夹下创建 `__init__.py` 文件，用于存放 Bundle 的代码；在 `tests` 文件夹下创建 `test_smoke.py` 文件，用于存放这次练习的验证代码。

练习内容

本次练习中，需要创建指定的 Bundle 类对 DUT 进行封装，然后编写一些新的测试用例：

1. 请根据**端口功能**，封装 FIFO 的端口：
 - 创建 `WriteBundle` 类，包含**向FIFO 写入数据**的相关端口。
 - 创建 `ReadBundle` 类，包含**从FIFO读取数据**的相关端口。
 - 创建 `InternalBundle` 类，用于封装 FIFO 内部状态信号。
 - 如果你觉得还需要补充其他 Bundle，请自行实现。
2. 为 `ReadBundle` 添加 `dequeue` 方法：实现方法 `dequeue(self)`，该方法执行一次读取操作。
3. 为 `WriteBundle` 添加 `enqueue` 方法：实现方法 `enqueue(self, data)`，该方法执行一次写入操作。
4. 使用封装好的 Bundle 环境，按照先前的 `test_smoke_dut` 的测试过程，创建测试用例 `test_bundle`
5. 使用封装好的 Bundle 环境，编写测试用例 `test_full_empty`：
 - 先将 `fifo` 装满后，判断 `full_o` 信号是否为 1
 - 再清空 `fifo`，判断 `empty_o` 信号是否为 1

- 执行清空 fifo 时，检查出队序列是否与入队序列一致

3. 使用 Agent 进一步封装

准备工作

这次练习相关代码存放到 `agent/__init__.py` 中

练习内容

本次练习需要给 DUT 编写一个 Agent，然后再利用写好的 Agent 构建一些测试用例：

1. 编写 FIFOAgent 类：

- 初始化时接收 `ReadBundle` 、 `WriteBundle` 和 `InternalBundle` 。
- 实现一个 `reset` 驱动方法，用于执行 FIFO 的复位操作。
- 将在 `Bundle` 层面实现的 `enqueue` 和 `dequeue` 方法包装成 Agent 的驱动方法。

2. 使用 FIFOAgent 封装好的参数，按照 `test_smoke_dut` 的测试过程，创建测试用例 `test_agent`

3. 思考：回顾你在 `Bundle` 和 `Agent` 练习中实现的 `enqueue` 和 `dequeue` 方法。将这些操作逻辑放在 `Bundle` 层实现（作为 `Bundle` 的方法），然后在 `Agent` 层简单调用 `@driver_method` 包装一下是更合适，还是直接在 `Agent` 的 `@driver_method` 中编写完整的信号操作逻辑更合适？或者有其他更好的方式？为什么？（提示：考虑代码复用、职责分离、抽象层次）

4. 思考：当我们把不同功能的输入封装为多个驱动函数之后，似乎每次只能串行一个驱动函数，对于多个功能同时输入的情况该怎么办呢？请阅读 toffee 文档中[如何同时调用多个驱动函数](#)部分，探

索 `Executor` 的基本用法, 然后在 `Agent` 中完成对**同时进行读写操作**的封装。(提示: 文档中 `Env` 不影响理解, 后续会进行介绍)。

4. 收集功能覆盖率

准备工作

- 确保你已经理解了同步 FIFO 的设计规范。
- 确保你的 FIFO `Agent` 和 `Bundle` 代码是可用的。
- 创建一个 `coverage` 文件夹 (或在你现有的 `env` 文件夹下), 用于存放覆盖率模型定义代码 (例如, `fifo_coverage.py`)。

练习内容

收集功能覆盖率并进行分析:

1. **拆分 SyncFIFO 功能点和测试点:** 根据你对同步 FIFO 规格的理解 (或参考标准 FIFO 行为), 列出其关键功能点。例如:
 - 基本操作: 成功写入、成功读取、无操作。
 - 边界状态: FIFO 空时尝试读取、FIFO 满时尝试写入、FIFO 空时写入、FIFO 满时读取。
 - 指针行为: 写指针追上读指针 (写满)、读指针追上写指针 (读空)、指针回绕。
 - 数据完整性: 写入的数据与后续读出的数据一致。
 - 复位行为: 复位后 FIFO 应为空, 指针复位。对于每个功能点, 思考需要哪些测试点 (输入激励条件) 来验证它, 以及需要观察哪些信号 (覆盖点) 来确认功能点被覆盖。

2. **编写 SyncFIFO 功能覆盖率模型:**

- 为上述拆分出的功能点创建对应的 `CovGroup` 函数（例如 `get_cover_group_boundary_operations`）。
- 在每个 `CovGroup` 函数内部，使用 `add_watch_point` 添加覆盖点和 Bins 来衡量对应的测试点是否被覆盖。利用之前介绍的比较函数、检查函数、工厂函数+Enum 等技巧。
- 思考每个 `CovGroup` 应该采用哪种采样方式（周期性采样还是基于特定序列的手动采样），并在 fixture 中进行相应的设置。

3. 查看报告并尝试提高功能覆盖率：

- 运行包含覆盖率收集的测试用例，并查看生成的覆盖率报告（如果不熟悉，请查阅 Toffee 的文档或新手教程第三讲）
- 分析报告，找出未覆盖的 Bins
- 尝试添加新的测试用例来提高覆盖率，但不要求功能覆盖率达到 100%，有提升就行

5. 编写参考模型并打包验证环境

准备工作

创建文件夹及文件：

- `env/__init__.py`：存放验证环境 Env
- `ref/__init__.py`：存放参考模型的代码

练习内容

编写参考模型并打包验证环境：

1. 编写同步 FIFO 的参考模型，并集成到验证环境中
2. 让功能覆盖率达到 100%